

The diagram illustrates the MetaSonic manifest reload and hot-swap interaction graph, organized into several functional blocks:

- Inputs and app catalog:** Contains initial inputs like `demoManifestReloadCatalog`, `ManifestReloadRequest`, and `AuthoringManifestDoc`.
- Current diagnostic / smoke entry points:** Includes command-line flags such as `--manifest-session-smoke`, `--manifest-reload-plan`, and `--manifest-stopped-audio-reload-smoke`.
- Pure manifest reload planning:** A central planning stage involving `planManifestReload`, `ManifestReloadPlan`, `manifestSessionOwnerOptions`, and `manifestReloadCommand`.
- Construction-time session setup:** A green-bordered section showing the setup of a new session, including `constructManifestSessionFromPlan`, `withSessionOwner / acquireSessionOwner`, `newRTGraphAdapter`, and `installSessionGraph`.
- Existing command-level hot-swap runtime path:** A cyan-bordered section detailing the runtime process, including `RTGraphAdapter.runHotSwap`, `admitSessionCommand`, and various fallback and preserving paths.
- Current owner, queue, service, and audio state:** An orange-bordered section managing the state during a reload, involving `SessionFanInService`, `SessionFanInHost`, `SessionOwner`, and `SessionCommandQueue`.
- Landed stopped-audio manifest reload:** A red-bordered section at the bottom showing the final steps of a reload, including `reloadManifestStoppedAudioHost`, `manifestReloadHostOps`, and `orchestrateHostStoppedAudioReload`.

The graph uses solid arrows for primary data flow and dashed arrows for conditional or fallback paths. Colored boxes highlight specific phases of the process, and various labels like "Not landed yet", "future input", and "future locked step" indicate the state of different components.